

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
MAZATENANGO SUCHITEPÉQUEZ
ANÁLISIS Y DISEÑO DE SISTEMAS II
ING. ANGEL ATILIO MALTEZ C.



PLANIFICACIÓN, ARQUITECTURA, COMPONENTES, UX Y PROTOTIPO

GILMAR TELLO VÁSQUEZ
CARNET: 3090-23-17613
8VO. SEMESTRE

SEPTIEMBRE DE 2026

INTRODUCCION

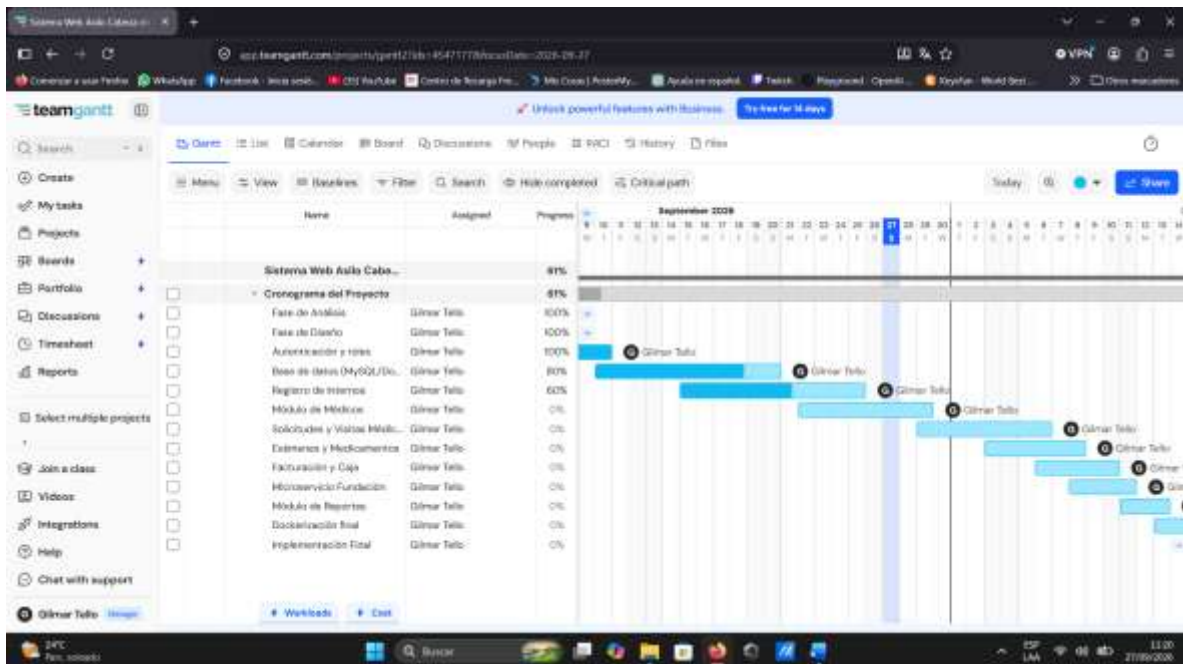
El presente documento amplía la fase de Análisis y Diseño ya entregadas, cubriendo cinco entregables solicitados para esta etapa del curso: la planificación real de trabajo, la arquitectura de software a emplear, la documentación de los componentes basados en clases, el diseño de experiencia de usuario con sus respectivas clases de interfaz, y el prototipo funcional del sistema.

PLANIFICACIÓN DE TRABAJO

La planificación se elaboró bajo un enfoque incremental: cada módulo del sistema se desarrolla, se conecta a la base de datos y se prueba antes de iniciar el siguiente, respetando las dependencias funcionales (por ejemplo, el módulo de Médicos depende de `tbl_usuarios`, ya completado en Autenticación). El recurso humano es el mismo en todas las actividades (yo, como analista y desarrollador único del proyecto), por lo que el tiempo estimado por tarea considera jornadas de desarrollo individuales de entre 3 y 5 días hábiles.

DIAGRAMA DE GANTT

El siguiente diagrama es una herramienta de gestión de proyectos de software, mostrando el estado real de avance a la fecha de esta entrega desarrollado en teamgantt.



No puedo compartir el link ya que estoy usando la versión gratuita, pero adjunto captura.

DETALLE DE ACTIVIDADES, RECURSOS Y TIEMPOS

Fase / Actividad	Descripción	Recursos	Fechas	Estado
Fase de Análisis	Levantamiento de requerimientos, reglas de negocio, casos de uso, diagramas UML.	Tello (analista/ desarrollador)	Entregado 07/08/2026	Completado (100%)
Fase de Diseño	Diseño de base de datos, arquitectura MVC + microservicios, evaluación de usabilidad del módulo de Registro de Internos.	Tello	Entregado 28/08/2026	Completado (100%)
Desarrollo - Autenticación	Login con roles (administrador, médico, médico especialista, laboratorio, farmacia, finanzas), sesiones con express-session, middlewares de control de acceso.	Tello - Node.js Express	01/09 - 10/09/2026	Completado (100%)
Desarrollo - Base de datos	Diseño e implementación de tbl_usuarios (roles ENUM, estado), esquema versionado en base/esquema.sql, migración de	Tello - MySQL Docker	10/09 - 20/09/2026	En curso (80%)

		XAMPP/MariaDB a Docker.			
Desarrollo de Registro Internos	-	Formulario de internos con validaciones (DPI, teléfono), conexión a MySQL (actualmente en localStorage).	Tello - Node.js / EJS	15/09 - 25/09/2026	En curso (60%)
Desarrollo de Módulo Médicos	-	Registro de médicos por el administrador (usa tbl_usuarios con título/especialidad); médico especialista.	Tello	22/09 - 29/09/2026	Pendiente
Desarrollo de Solicitudes y Visitas Médicas	-	Creación de solicitudes con notificación automática por correo (Nodemailer) y registro de visitas médicas.	Tello - Nodemailer	29/09 - 06/10/2026	Pendiente
Desarrollo de Exámenes y Medicamentos	-	Módulos de laboratorio (exámenes) y farmacia (medicamentos).	Tello	03/10 - 08/10/2026	Pendiente
Desarrollo de Facturación y Caja	-	Entradas/salidas/caja: donaciones, cuotas familiares, gastos, pagos a la fundación; tbl_pagos, CuotaMensual.	Tello	06/10 - 10/10/2026	Pendiente

Desarrollo Microservicio Fundación	- Servicio independiente (puerto 3002) que se comunica vía API con asilo_web.	Tello Node.js / API REST	- 08/10 - 11/10/2026	Pendiente
Desarrollo Reportes	- Módulo de 7 reportes administrativos y clínicos.	Tello	11/10 - 13/10/2026	Pendiente
Dockerización final	Contenedores de app + base de datos, variables de entorno (.env), despliegue con acceso público.	Tello - Docker / Docker Compose	13/10 - 15/10/2026	Pendiente
Implementación Final	Entrega, pruebas finales y documentación de cierre.	Tello	16/10/2026	Pendiente

METODOLOGÍA DE SEGUIMIENTO

Control semanal de avance

Cada viernes se revisa el porcentaje completado de la actividad activa contra lo planificado.

Los módulos ya completados (Análisis, Diseño, Autenticación) sirven de línea base para reestimar los módulos pendientes si surgen retrasos.

Las actividades del microservicio de la Fundación y de Reportes dependen de que Facturación y Caja estén completos, por lo que se planificaron en secuencia y no en paralelo, dado que el proyecto se desarrolla en solitario.

El margen entre la fecha de Dockerización (15/10) y la Implementación Final (16/10) se reserva como colchón de contingencia para corregir errores de último momento.

ARQUITECTURA A EMPLEAR

Estilo arquitectónico

Patrón MVC (Modelo–Vista–Controlador) para el sistema principal asilo_web: separa la persistencia (modelos), la lógica de negocio (controladores) y la presentación (vistas EJS), facilitando el mantenimiento individual de cada capa.

Arquitectura de microservicios: el sistema principal, el servicio de cálculo de costos (puerto 3000) y el servicio de la Fundación (puerto 3002) son componentes independientes que se comunican vía API REST, cada uno con su propio ciclo de despliegue.

Contenerización con Docker: cada servicio (aplicación y base de datos) corre en su propio contenedor, orquestados con Docker Compose, garantizando que el entorno de desarrollo sea idéntico al de producción.

Componentes principales

asilo_web (Node.js + Express + EJS): gestiona autenticación, roles, internos, médicos, solicitudes, visitas, exámenes, medicamentos, facturación y reportes.

Microservicio de cálculo de costos: recibe costos de consulta/examen/medicamento y aplica el 20% de descuento de fundación, devolviendo subtotal, descuento y total en JSON.

Microservicio de Fundación: administra la información de la fundación de forma desacoplada del sistema principal, evitando que un cambio en uno afecte al otro.

Base de datos MySQL (asilo_db): contenedor independiente, con esquema versionado en el repositorio (base/esquema.sql) para poder recrearla ante fallos, como el ocurrido con la instancia de XAMPP/MariaDB.

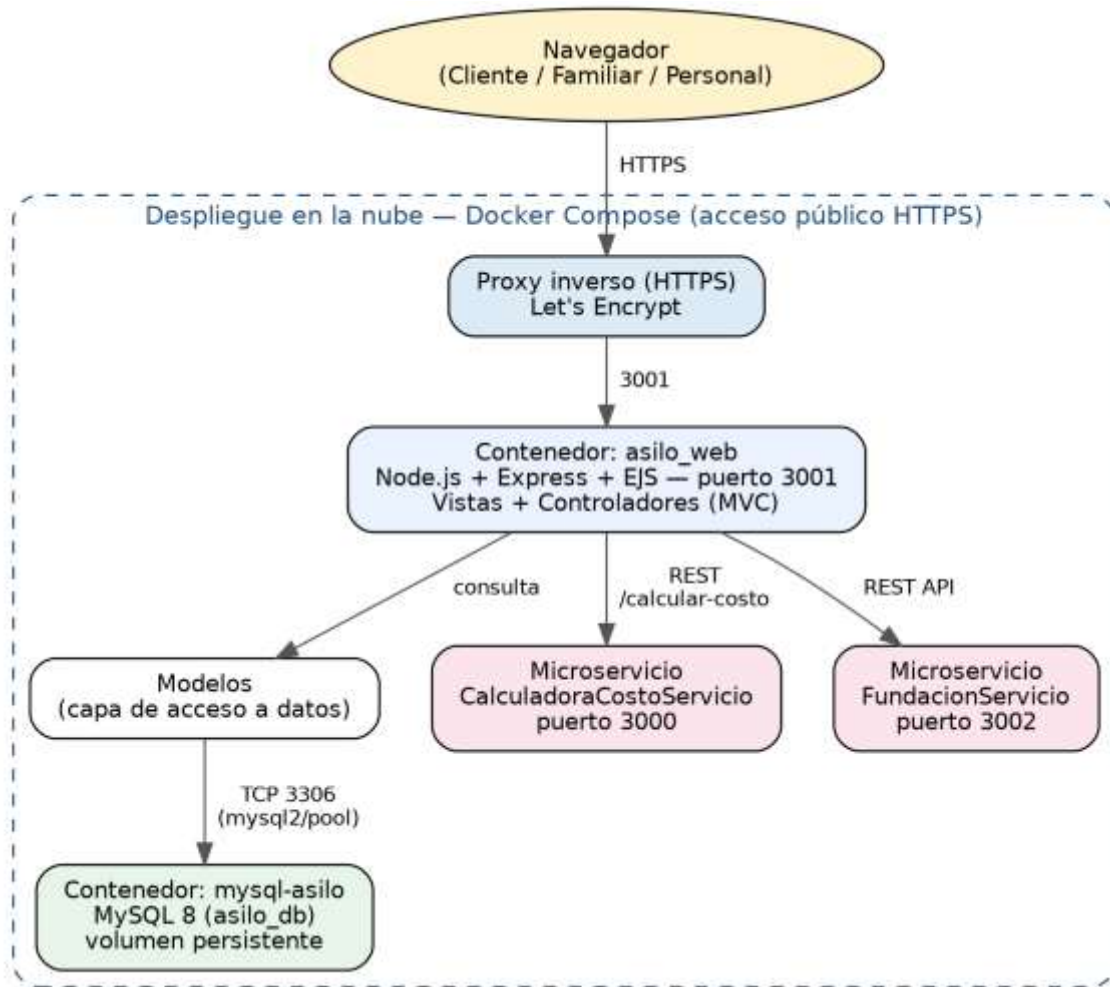
Justificación de la arquitectura

Se eligió MVC combinado con microservicios porque el dominio del asilo tiene un núcleo administrativo estable (usuarios, internos, historial clínico) y funciones periféricas que cambian con reglas propias y volumen de uso distinto: el cálculo de costos es una operación de negocio pura y reutilizable desde cualquier módulo, y la Fundación es una entidad externa que eventualmente podría gestionarse desde otro sistema o convertirse en un servicio compartido con otros asilos de la misma fundación. Separarlos como microservicios evita acoplar esa lógica al monolito principal y permite escalarlos o reemplazarlos de forma independiente.

Diagrama de despliegue

El diagrama de despliegue (elaborado en PlantUML durante la fase de Análisis) describe tres contenedores Docker: asilo_web (Node/Express, puerto 3001), mysql-asilo (base de datos, volumen persistente) y los microservicios de costos y fundación, todos dentro de una misma red Docker interna (docker-network) y expuestos hacia el exterior únicamente a través del contenedor principal y su proxy inverso.

DIAGRAMA DE ARQUITECTURA Y DESPLIEGUE



Estrategia de despliegue y acceso público

Para exponer el sistema con acceso público se recomienda desplegar los contenedores Docker en un proveedor en la nube (por ejemplo, una VM gratuita/de bajo costo en Oracle Cloud, Railway o Render), asignando un dominio o subdominio y certificado HTTPS mediante Let's Encrypt. Esta opción evita depender de la conexión residencial y de la configuración de redirección de puertos (port forwarding) y DNS dinámico que exigiría un despliegue casero. Como alternativa de contingencia, el mismo docker-compose puede ejecutarse localmente y exponerse temporalmente mediante un túnel (por ejemplo ngrok o Cloudflare Tunnel) si la nube no está disponible antes de la entrega.

Consideraciones de seguridad

Contraseñas cifradas con `bcryptjs` antes de almacenarse en `tbl_usuarios`.

Sesiones de servidor con `express-session`; las rutas protegidas verifican `req.session.usuario` en cada petición.

Variables sensibles (credenciales de base de datos, claves de sesión) fuera del código fuente, en un archivo `.env`, para no exponerlas al dockerizar ni subirlas al repositorio.

Control de acceso basado en roles (RBAC) a nivel de middleware, evitando que un usuario sin privilegios de administrador acceda a rutas restringidas aunque conozca la URL.

DOCUMENTACIÓN DE COMPONENTES DE SOFTWARE (BASADOS EN CLASES)

Los siguientes componentes se derivan del modelo de clases y del esquema de base de datos ya definidos en la fase de Análisis (diagramas de clases y objetos en PlantUML) y de la implementación actual del sistema.

Clase / Componente	Capa (MVC)	Responsabilidad
Usuario	Modelo	Representa a cualquier persona con acceso al sistema (admin, médico, laboratorio, farmacia, finanzas). Los médicos NO tienen tabla propia: título y especialidad se guardan en el mismo tbl_usuarios.
Interno	Modelo	Datos del adulto mayor registrado en el asilo: identificación, datos personales, parentesco del familiar responsable.
Solicitud	Modelo	Solicitud generada para un interno (p. ej. atención médica); dispara notificación automática por correo al familiar.
VisitaMedica	Modelo	Registro de una visita/consulta médica realizada a un interno.
Examen	Modelo	Examen de laboratorio solicitado/realizado a un interno.
Medicamento	Modelo	Medicamento recetado/dispensado por farmacia.
Pago / CuotaMensual	Modelo	Registro de pagos y cuotas mensuales familiares (tbl_pagos).
Psicopatologia	Modelo	Condición psicológica/psiquiátrica asociada a un interno.

AutenticacionControlador	Controlador	Login, logout, manejo de sesión (express-session) y verificación de credenciales.
InternoControlador	Controlador	CRUD de internos; validación de datos antes de persistir.
Middlewares (requiereLogin, requiereAdmin)	Controlador (transversal)	Cadena de verificación de acceso antes de llegar al controlador destino.
CalculadoraCostoServicio	Componente / Microservicio (puerto 3000)	Recibe costos de consulta, examen y medicamento; aplica 20% de descuento fundación; retorna subtotal, descuento y total en JSON.
FundacionServicio	Componente / Microservicio (puerto 3002)	Gestiona la información de la fundación como servicio independiente, comunicado por API con asilo_web.
Vistas EJS (login, dashboard, internos, parciales)	Vista	Renderizado de la interfaz; los parciales (cabecera, menu, pie) se reutilizan en todas las pantallas autenticadas.

Detalle de clases clave

Usuario - atributos: idUsuario, nombre, usuario, contrasena (hash), rol (ENUM), estado (activo/desactivado), titulo, especialidad (nulos si no aplica). Métodos: autenticar(), cambiarEstado(), obtenerPorRol().

Interno - atributos: idInterno, nombres, apellidos, sexo, fechaNacimiento, dpi, parentescoResponsable, telefonoResponsable. Métodos: registrar(), actualizar(), listarActivos().

Solicitud - atributos: idSolicitud, idInterno, tipo, fechaCreacion, estado. Métodos: crear() (dispara notificarPorCorreo()), cambiarEstado().

CalculadoraCostoServicio - atributo: porcentajeDescuentoFundacion = 20%.
Método: calcularCosto(consulta, examen, medicamento) - { subtotal, descuento, total }.

Relaciones principales

Usuario 1-N Interno (un médico atiende muchos internos; el administrador registra muchos usuarios).

Interno 1-N Solicitud, 1-N VisitaMedica, 1-N Examen, 1-N Medicamento.

Interno 1-N Pago / CuotaMensual (relación con Facturación y Caja).

Interno 0..1 Psicopatologia (relación opcional, no todo interno tiene una condición registrada).

EXPERIENCIA DE USUARIO (UX) - PATRONES Y DISEÑO

Patrones de diseño aplicados

MVC: separación de datos (modelos), lógica (controladores) y presentación (vistas EJS).

Chain of Responsibility: los middlewares `requiereLogin` y `requiereAdmin` se encadenan antes del controlador (`requiereLogin`, `requiereAdmin`, `handler`), permitiendo interceptar peticiones no autorizadas sin duplicar validaciones en cada ruta.

Composite / Template en las vistas: las plantillas parciales (`cabecera.ejs`, `menu.ejs`, `pie.ejs`) se combinan en cada vista para mantener consistencia visual y evitar duplicación de código.

Diseño responsivo mobile-first con Bootstrap 5: el menú lateral usa `offcanvas-md` `offcanvas-start` (fijo en escritorio, deslizable en móvil).

Principios de usabilidad (Nielsen) aplicados tras la evaluación de agosto

Visibilidad del estado del sistema: retroalimentación visual al guardar formularios (éxito/error).

Prevención de errores: validaciones de formato (patrón de 13 dígitos para DPI, 8 dígitos para teléfono, campos requeridos, selects en vez de texto libre para sexo/parentesco).

Consistencia y estándares: mismo sistema de diseño (variables CSS `--azul`, `--texto`, `--borde`, etc.) en todas las pantallas autenticadas.

Reconocer antes que recordar: el menú lateral siempre visible (o accesible con un toque en móvil) evita que el usuario deba memorizar rutas de navegación.

Flujo de navegación por rol

Administrador: Login - Dashboard (menú completo) - Usuarios/Médicos, Internos, Reportes, Configuración.

Médico / Médico especialista: Login - Dashboard (menú clínico) - Internos asignados, Visitas Médicas, Solicitudes.

Laboratorio: Login - Dashboard (menú de exámenes) - Registro y resultado de exámenes.

Farmacia: Login - Dashboard (menú de medicamentos) - Entrega y control de medicamentos.

Finanzas: Login - Dashboard (menú financiero) - Caja, Pagos, Cuotas mensuales, Reportes de costos.

Clases de interfaz - Frontend

LoginView (login.ejs): formulario de acceso, no usa las plantillas compartidas.

DashboardView (dashboard.ejs): panel dinámico que renderiza un menú distinto según el rol del usuario.

InternoFormView: formulario de registro de internos con validación en cliente (JS) y en servidor.

Parciales compartidos: CabeceraView, MenuLateralView, PieView, reutilizados por todas las vistas protegidas.

Clases de interfaz - Backend

AutenticacionControlador: expone /login (GET/POST) y /logout.

InternoControlador: expone las rutas bajo /internos (registro, listado, edición).

Middlewares de autenticación: interfaz transversal que protege cualquier ruta que la declare.

CalculadoraCostoServicio: expone POST /calcular-costo como interfaz HTTP/JSON independiente.

PROTOTIPO

El prototipo funcional corresponde al sistema actualmente implementado, con las siguientes pantallas operativas:

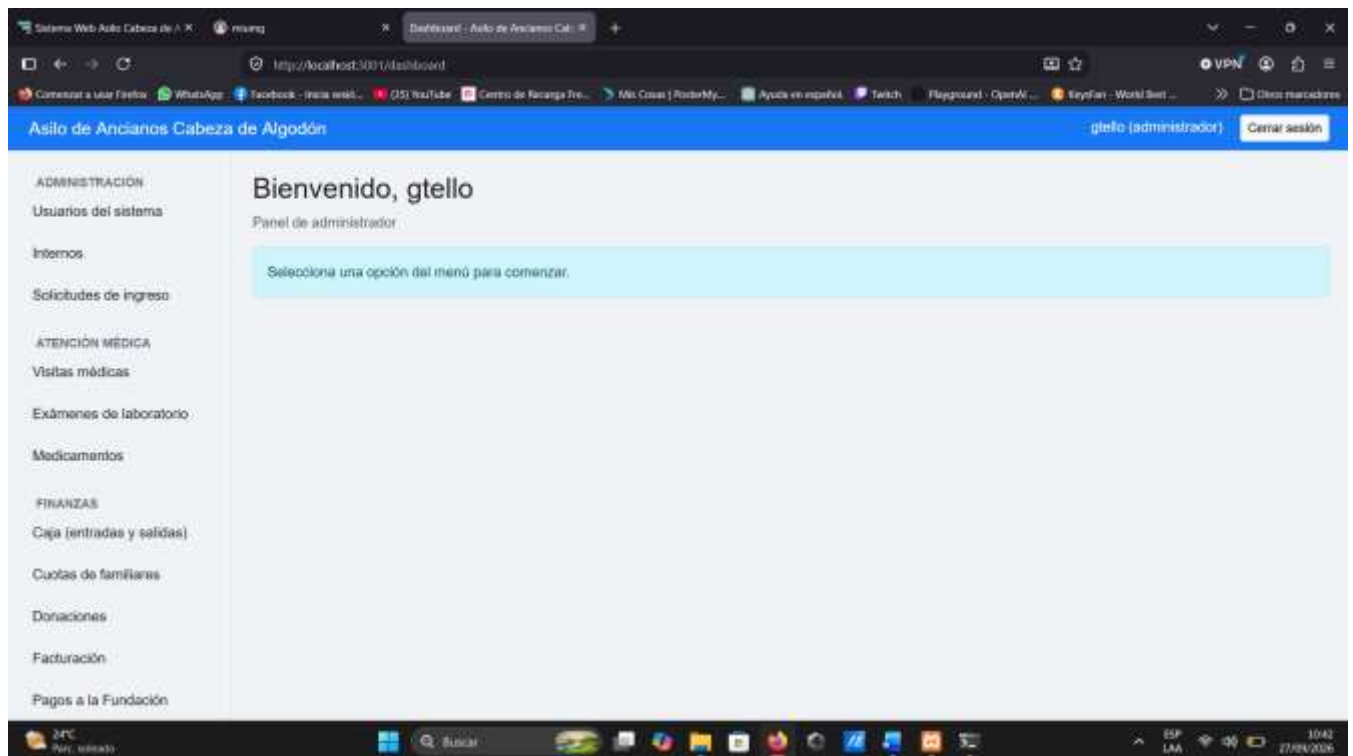
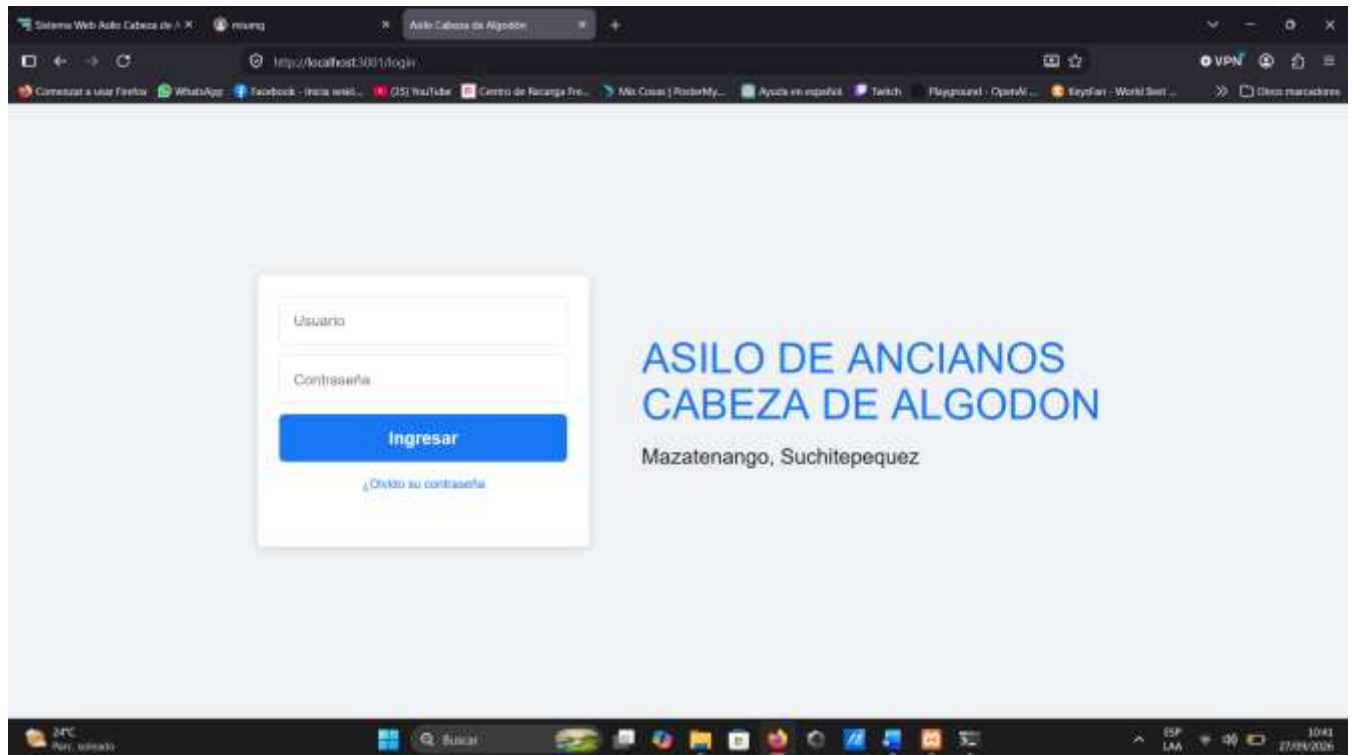
Pantalla de login con diseño tipo Facebook (formulario a la izquierda, identidad del asilo a la derecha; se invierte en móvil).

Dashboard con menú dinámico según rol (administrador, médico, médico especialista, laboratorio, farmacia, finanzas).

Formulario de registro de internos con validaciones de formato (DPI, teléfono) y feedback visual inmediato.

Middleware de sesión probado: intentos de acceso directo a rutas protegidas sin sesión redirigen automáticamente a /login.

Microservicio de cálculo de costos probado con Thunder Client, aplicando el 20% de descuento de fundación sobre casos reales de consulta+examen+medicamento.



Asilo de Ancianos Cabeza de Algodón

gileto (administrador)Cerrar sesión

ADMINISTRACIÓN

Usuarios del sistema

Internos

Solicitudes de ingreso

ATENCIÓN MÉDICA

Visitas médicas

Exámenes de laboratorio

Medicamentos

FINANZAS

Caja (entradas y salidas)

Cuotas de familiares

Donaciones

Facturación

Pagos a la Fundación

Usuarios

+ Nuevo usuario

Buscar por nombre, apellido, DPI, usuario o correo

Buscar

Nombre completo	DPI	Usuario	Rol	Estado	Acciones
Gilmar Teño	2558846561001	gileto	administrador	Activo	EditarDesactivar

1 usuario(s) encontrado(s)

100°C

Buscar

ESP

LAA

100%

27/09/2024